



**GIK Institute of Engineering Sciences and Technology,
Pakistan**

Department of Computer Engineering
Computer System Architecture

INDIVIDUAL PROJECT REPORT

Design and Verification of a Single-Cycle RISC-V Processor

Mueed Rauf

Submitted to: Lab Engineer Sir Abbas

Course: Computer System Architecture (CSA)

Project Name: Single-Cycle RISC-V Processor Design

June 21, 2026

Acknowledgements

I would like to express my sincere gratitude to Professor Fahad and Sir Abbas for their guidance, feedback, and encouragement throughout this project. Their direction helped shape the design choices, verification strategy, and final presentation of this single-cycle processor.

I also thank the Computer System Architecture (CSA) course and the USTP program for providing the academic foundation for this work. The project was completed as an individual submission, and the learning experience gained from designing, implementing, and testing each module was valuable and rewarding.

Contents

List of Figures	iv
List of Tables	v
List of Diagrams and Simulations	vi
1 Introduction	1
1.1 Project Overview	1
1.2 Objectives	1
1.3 Report Organization	2
2 RISC-V Background	3
2.1 RISC-V Overview	3
2.2 Instruction Classes Used in the Project	3
2.3 Why a Single-Cycle Design	3
2.4 Instruction Coverage	4
2.5 Design Focus	4
3 Processor Design	5
3.1 Overall Architecture	5
3.2 Datapath Control	6
3.3 Execution Path	6
3.4 Instruction Formats	6
3.5 Control and Data Path Summary	7
3.6 Module Summary	7
3.7 Instruction Flow	9
4 Module Verification and Simulations	10
4.1 Verification Strategy	10
4.2 Module-Level Results	10
4.2.1 Program Counter	10
4.2.2 Instruction Memory	11
4.2.3 Decoder	11
4.2.4 Control Unit	12
4.2.5 ALU Decoder	12
4.2.6 ALU	13
4.2.7 Register File	14
4.2.8 Sign Extender	14
4.2.9 Data Memory	14
4.2.10 Top-Level Processor	15

5	Discussion	16
5.1	Design Observations	16
5.2	Strengths	16
5.3	Limitations	16
5.4	Future Work	17
6	Conclusions	18
	Bibliography	19
	Appendices	20
A	Appendix Title	21
A.1	Simulation File List	21
A.2	Project Note	21

List of Figures

Figure 3.1	Single-cycle processor datapath.	5
Figure 4.1	Program counter simulation.	11
Figure 4.2	Instruction memory simulation.	11
Figure 4.3	Decoder simulation.	12
Figure 4.4	Control unit simulation.	12
Figure 4.5	ALU decoder simulation.	13
Figure 4.6	ALU simulation.	13
Figure 4.7	Register file simulation.	14
Figure 4.8	Sign extender simulation.	14
Figure 4.9	Data memory simulation.	15
Figure 4.10	Top-level processor simulation.	15

List of Tables

Table 3.1	Instruction formats used in the processor	6
Table 3.2	Control behavior for supported instructions	7
Table 3.3	Summary of processor modules and their roles	8

List of Diagrams and Simulations

This section is intentionally brief. The report now places each available simulation image directly under its own heading, so the reader can move from explanation to evidence without needing a separate source table.

Chapter 1

Introduction

1.1 Project Overview

This report presents the design and verification of an individual single-cycle processor implemented in SystemVerilog. The processor supports a focused subset of RISC-V-style instructions, including R-type operations, load and store instructions, immediate arithmetic through `addi`, conditional branching through `beq`, and jump-and-link through `jal`.

The design is organized as a set of small, testable modules connected by a top-level processor module. Each module has a dedicated testbench in the simulation directory, which makes it possible to verify the datapath and control logic both independently and as an integrated system.

1.2 Objectives

The main objectives of the project are to:

- build a working single-cycle processor with clear module boundaries,
- implement instruction fetch, decode, execute, memory, and write-back in one clock cycle,
- support the required instruction types for coursework-level processor design,
- verify every hardware block with a separate simulation testbench, and
- present the final design in a clear report suitable for the CSA course.

1.3 Report Organization

Chapter 2 introduces the RISC-V background used in this project and relates it to the single-cycle implementation style described by Harris and Harris. Chapter 3 explains the overall processor design and the role of each module. Chapter 4 presents the available module-level simulations and the top-level verification plan. Chapters 5 and 6 discuss the design trade-offs and conclude the report.

Chapter 2

RISC-V Background

2.1 RISC-V Overview

This project follows the RISC-V style described by Harris and Harris in their book on digital design and computer architecture [1]. RISC-V is a reduced instruction set architecture with a clean encoding style and a small set of core ideas that make it well suited to educational processor design. The instruction formats used in this project are intentionally limited so that the datapath can remain compact and easy to explain.

2.2 Instruction Classes Used in the Project

The implementation supports a practical subset of RISC-V-style instructions. R-type instructions are used for register-to-register arithmetic and logic. Load and store instructions demonstrate the memory interface. The immediate arithmetic instruction `addi` shows how constant operands are extended into the datapath. Branch instructions show how the PC can be redirected based on a comparison result, and `jal` shows how a control-transfer instruction can both change the PC and save the return address.

2.3 Why a Single-Cycle Design

A single-cycle datapath is a useful teaching model because it exposes the full flow of an instruction without introducing pipeline hazards or inter-stage registers. The design is slower than a pipelined processor, but it is much easier to verify at the module level and much easier to document in a report. That makes it a strong match for a course project where correctness and clarity are more important than performance.

2.4 Instruction Coverage

The implemented subset covers the most common datapath patterns used in introductory processor projects:

- R-type instructions exercise register-to-register ALU operations.
- `lw` and `sw` exercise data memory read and write paths.
- `addi` exercises immediate generation and ALU input selection.
- `beq` exercises comparison and branch target selection.
- `jal` exercises jump target generation and link register write-back.

2.5 Design Focus

The report emphasizes clarity over complexity. Rather than extending the processor into a pipelined or hazard-managed design, the implementation keeps the control path compact so that each module can be explained and tested independently.

3.2 Datapath Control

The top-level diagram highlights the main control signals that govern the processor. **PCWrite** enables the program counter update, **AdrSrc** selects between instruction fetch and data memory addressing, **IRWrite** controls instruction loading, and **MemWrite** enables store operations. The control unit also drives **ResultSrc**, **ALUControl**, **ALUSrcA**, **ALUSrcB**, **ImmSrc**, and **RegWrite** so the datapath can execute each supported instruction correctly.

3.3 Execution Path

For R-type instructions, the processor reads two registers, performs the selected ALU operation, and writes the result back to the register file. For load and store instructions, the ALU generates the memory address and the data memory provides or accepts the data word. For **addi**, the sign extender supplies the immediate operand to the ALU. For branch and jump instructions, the PC-selection logic uses the computed target address to redirect execution.

3.4 Instruction Formats

The implementation uses the standard RISC-V style instruction split to decode each supported operation. Table 3.1 summarizes the main fields used by the processor.

Table 3.1: Instruction formats used in the processor

Type	Main fields	Use in this project
R-type	op, rd, funct3, rs1, rs2, funct7	Used for register-to-register arithmetic and logic instructions.
I-type	op, rd, funct3, rs1, imm[11:0]	Used for lw and addi .
S-type	op, imm[11:5], rs2, rs1, funct3, imm[4:0]	Used for sw .

Type	Main fields	Use in this project
B-type	op, imm bits, rs2, rs1, funct3, imm bits	Used for <code>beq</code> .
J-type	op, rd, imm bits	Used for <code>jal</code> .

3.5 Control and Data Path Summary

Table 3.2 summarizes how the major control signals map to the supported instruction classes.

Table 3.2: Control behavior for supported instructions

Instruction	Main controls	Effect
R-type	RegWrite, ALUSrc=0, Re- sultSrc=ALU	Performs register-to-register computation.
<code>lw</code>	RegWrite, ALUSrc=1, Re- sultSrc=memory	Reads data from memory into a register.
<code>sw</code>	MemWrite, ALUSrc=1	Stores register data into memory.
<code>addi</code>	RegWrite, ALUSrc=1, Re- sultSrc=ALU	Adds an immediate to a register value.
<code>beq</code>	Branch, ALU subtraction	Changes the PC when the registers match.
<code>jal</code>	jump, RegWrite, ResultSrc=PC+4	Writes the return address and jumps.

3.6 Module Summary

Table 3.3 gives a concise description of the hardware blocks used in the design.

Table 3.3: Summary of processor modules and their roles

Module	Role in the processor
Program_Counter	Stores the current PC value and resets the processor to address zero.
adder	Computes $PC + 4$ for sequential execution and $PC + ImmExt$ for branch or jump targets.
inst_Mem	Supplies the instruction word from the internal instruction memory used in simulation.
Decoder	Splits the instruction into opcode, registers, funct fields, and immediate fragments.
Control_Unit	Generates all datapath control signals from the opcode and zero flag.
sign_extender	Expands load, store, branch, and jump immediates into 32-bit values.
ALU_decoder	Maps the control information and funct fields to the final ALU control code.
ALU	Performs arithmetic, logic, comparison, and zero-detection operations.
Reg_file	Implements the 32-register file with combinational reads and synchronous writes.
Data_Mem	Models the byte-addressed data memory used by load and store instructions.
mux2_1 and mux2_1_SrcB	Select the next PC source and the ALU B operand source.
mux4_1	Selects the write-back data between ALU result, memory data, and PC+4 for jal.
Processor	Integrates every module into the final top-level single-cycle processor.

3.7 Instruction Flow

An instruction begins at the program counter and passes through the instruction memory and decoder. The control unit interprets the opcode and produces the datapath configuration needed for that instruction. The register file then provides operands, the ALU performs the requested operation, and the result is either written back directly, stored in data memory, or used as the next PC.

Branch and jump instructions follow the same general flow, but they redirect the PC through the target adder and the PC-selection multiplexer. This keeps the implementation compact while still supporting control-flow changes.

Chapter 4

Module Verification and Simulations

4.1 Verification Strategy

Each hardware block in the project has its own testbench in the simulation folder. This allows the behavior of the block to be checked before it is connected to the complete processor. The top-level processor testbench then exercises the full instruction flow using the program stored in the instruction memory.

4.2 Module-Level Results

The following subsections are organized around the simulation screenshots already available for the individual blocks. Each one shows that the module behaves correctly before integration into the complete processor.

4.2.1 Program Counter

The program counter testbench confirms that reset initializes the PC to zero and that each rising clock edge loads the next address.

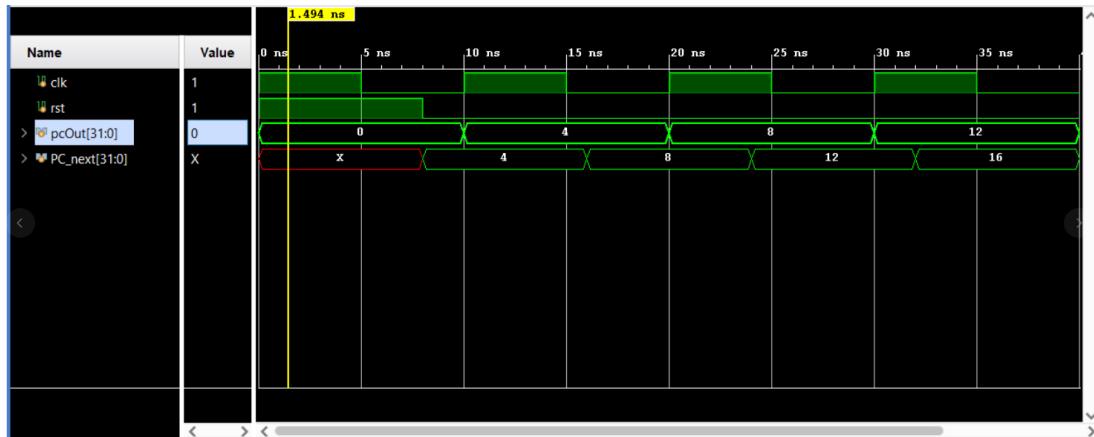


Figure 4.1: Program counter simulation.

4.2.2 Instruction Memory

The instruction memory testbench demonstrates that the stored program is fetched correctly based on the current PC value.

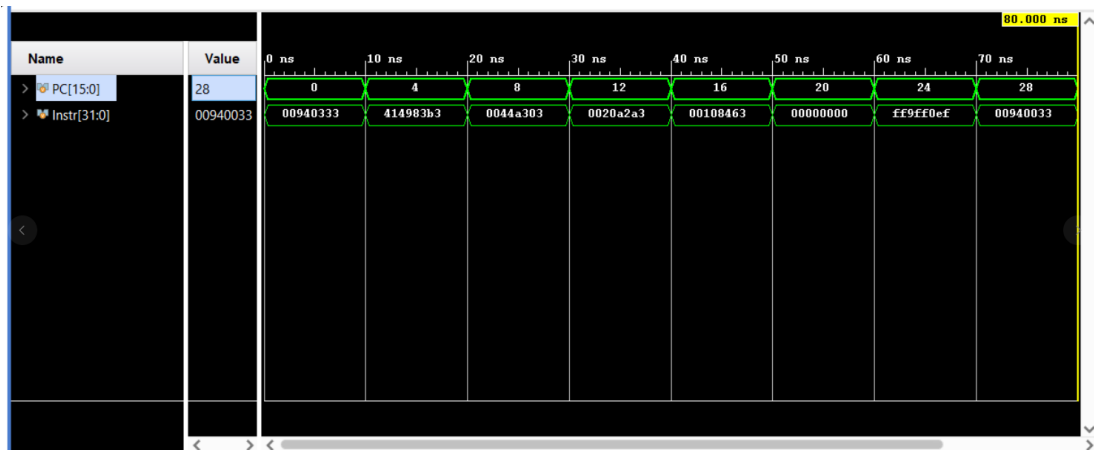


Figure 4.2: Instruction memory simulation.

4.2.3 Decoder

The decoder testbench checks that each instruction is separated into opcode, register indices, funct fields, and immediate fragments.

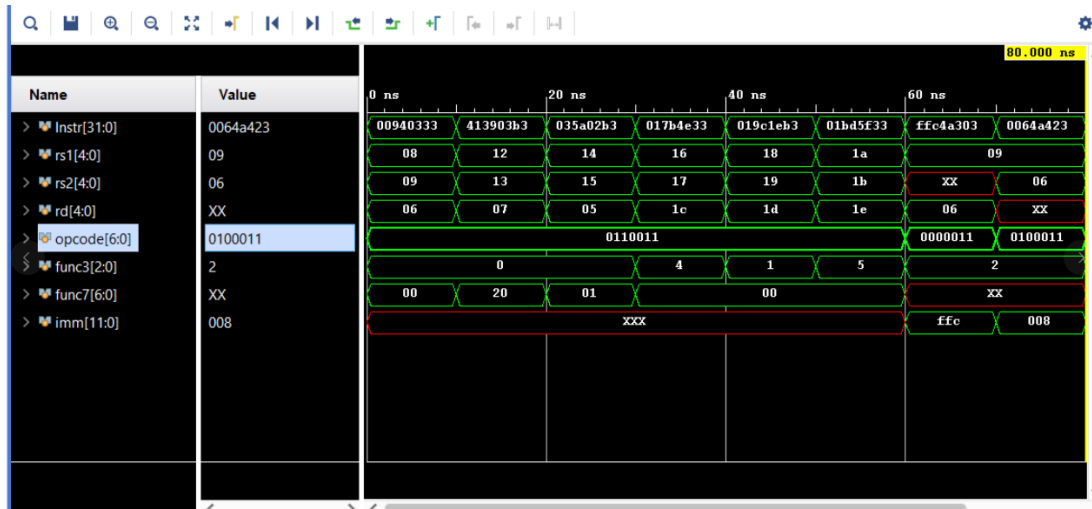


Figure 4.3: Decoder simulation.

4.2.4 Control Unit

The control unit testbench confirms that the opcode selects the correct control signals for every supported instruction class.



Figure 4.4: Control unit simulation.

4.2.5 ALU Decoder

The ALU decoder testbench verifies that the control logic and funct bits are translated into the correct final ALU control code.

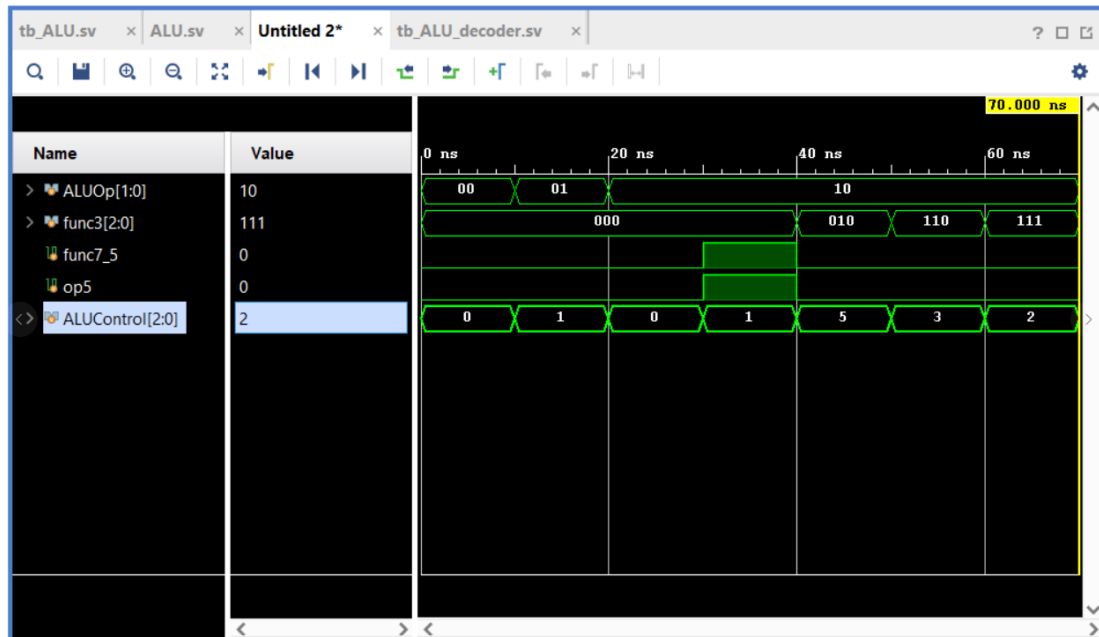


Figure 4.5: ALU decoder simulation.

4.2.6 ALU

The ALU testbench shows the arithmetic and logic outputs for the selected control codes. The screenshots confirm add, or, subtract, and comparison behavior, along with the zero flag output.

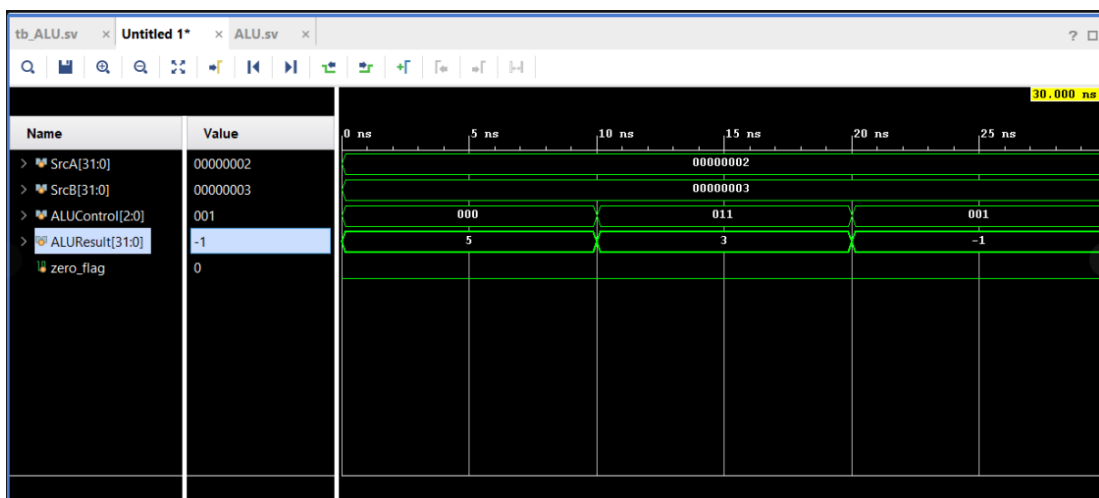


Figure 4.6: ALU simulation.

4.2.7 Register File

The register file testbench confirms that reads are available from the selected source registers and that write-back updates the destination register on the active clock edge.

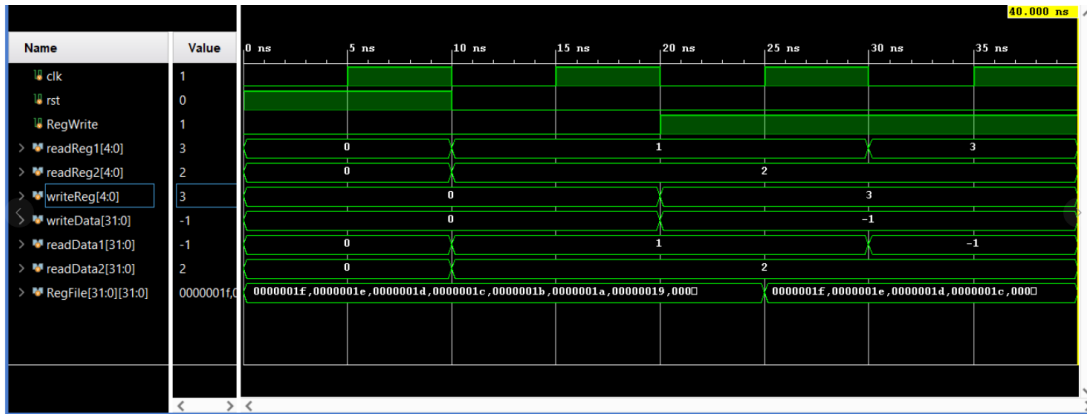


Figure 4.7: Register file simulation.

4.2.8 Sign Extender

The sign extender testbench verifies immediate formation for load, store, branch, and jump instructions.

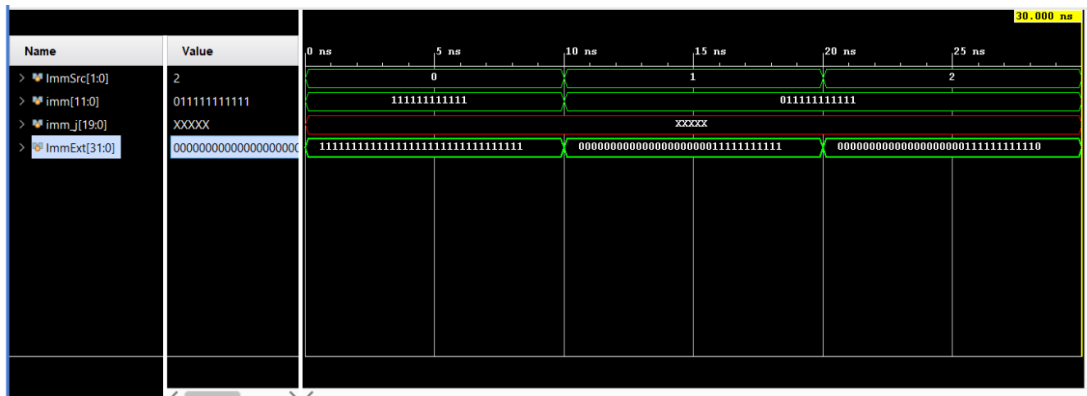


Figure 4.8: Sign extender simulation.

4.2.9 Data Memory

The data memory testbench checks the read and write path used by load and store instructions.

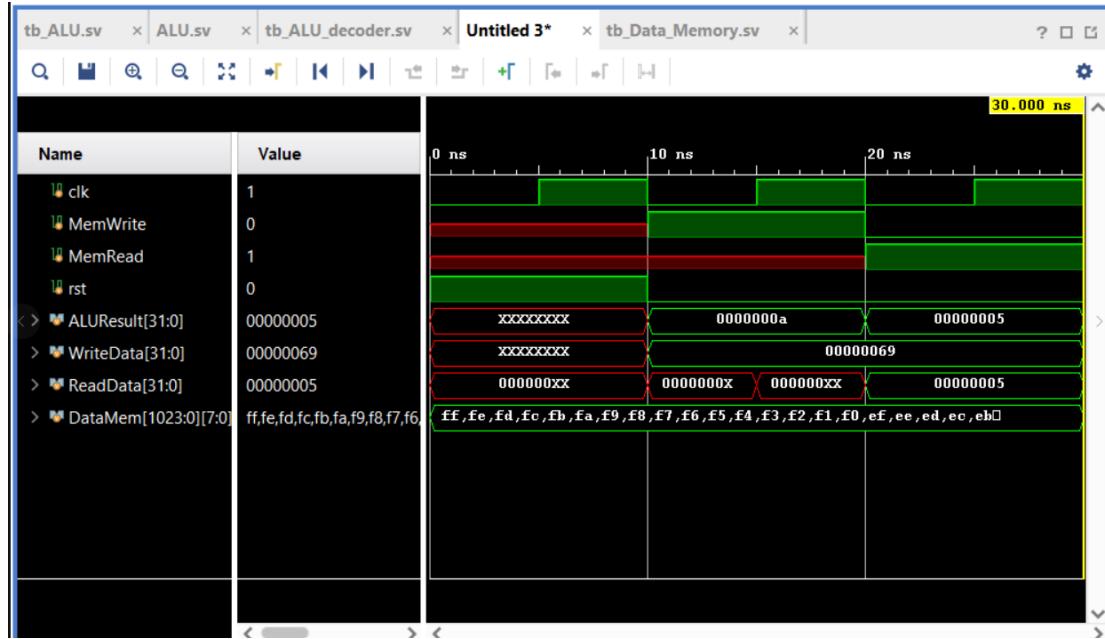


Figure 4.9: Data memory simulation.

4.2.10 Top-Level Processor

The final top-level architecture diagram and waveform are the main evidence that the processor works as a complete system. This section ties together the control unit, datapath, memory interface, and write-back path for the supported instruction sequence.

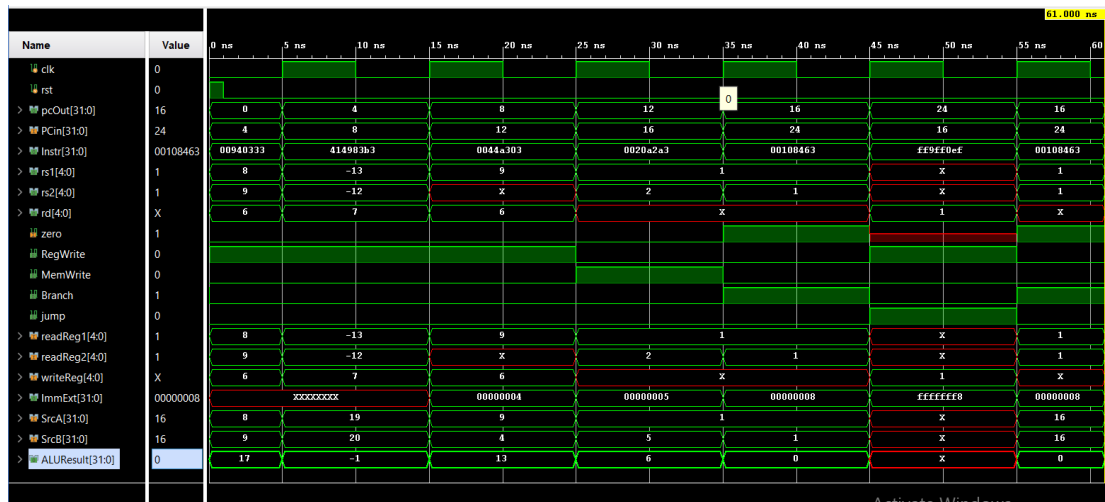


Figure 4.10: Top-level processor simulation.

Chapter 5

Discussion

5.1 Design Observations

The completed processor is easy to understand because each instruction travels through a visible and fixed datapath. That simplicity makes the design suitable for a course project and for simulation-based verification. The separation of responsibilities across modules also makes debugging much easier than in a single monolithic design.

5.2 Strengths

- The processor supports the required instruction classes in a clear and traceable way.
- The control unit is compact and directly reflects the opcode-driven behavior of the datapath.
- Individual testbenches make it possible to isolate design mistakes early.
- The top-level module demonstrates how the smaller blocks combine into a complete CPU.

5.3 Limitations

Because this is a single-cycle processor, the clock period must accommodate the slowest instruction path. That means the design is simpler than a pipelined implementation, but it does not aim for maximum performance. The current memory models are also intentionally lightweight and are best suited for simulation and coursework rather than a full hardware deployment.

5.4 Future Work

The design could be extended in several directions if the project is continued. A pipeline version would improve performance, additional arithmetic and logic instructions would broaden the instruction set, and a richer memory model would make the processor more realistic. It would also be useful to expand the verification environment with more extensive instruction-program tests.

Chapter 6

Conclusions

The project successfully organizes a single-cycle processor into clear, independently testable modules and connects them through a top-level datapath. The implementation supports the required instruction subset, including R-type instructions, load and store operations, immediate arithmetic, branching, and jump-and-link behavior.

The main value of the work is not only that the processor functions correctly in simulation, but also that the design is easy to explain and document. That makes it a strong fit for the CSA course, where the emphasis is on understanding processor organization and control flow.

In summary, the report shows how a complete processor can be built from small hardware blocks and verified systematically through simulation. With the planned addition of waveform images, the documentation will provide a complete picture of both module-level and system-level behavior.

Bibliography

- [1] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2021.

Appendices

Appendix A

Appendix Title

A.1 Simulation File List

The following testbenches are included with the project and were used to validate the design at module level and at system level:

- `tb_program_Counter.sv`
- `tb_ALU.sv`
- `tb_Control_Unit.sv`
- `tb_Data_Memory.sv`
- `tb_Decoder.sv`
- `tb_inst_Mem.sv`
- `tb_processor.sv`
- `tb_Reg_file.sv`
- `tb_Sign_Extender.sv`
- `tb_ALU_decoder.sv`

A.2 Project Note

This appendix can be expanded later with the instruction encoding table, additional waveforms, or any module-level screenshots that need to appear after the main report chapters.